

PRACTICAL WEEEEK 9

TASK-1:

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Ans:

Create a test file:

```
arikel@DESKTOP-AMUSJME:~/week5prac$ echo "This is a test file f  
or low level Linux file I/O." > source.txt  
arikel@DESKTOP-AMUSJME:~/week5prac$ cat source.txt  
This is a test file for low level Linux file I/O.
```

lowlevel_copy.c

arikela@DESKTOP-AMUSJME: ×

+ ▾

—

□

×

GNU nano 8.7.1

lowlevel_copy.c *

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    if (argc != 3)
    {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    src = open(argv[1], O_RDONLY);

    if (src == -1)
    {
        perror("open source");
        return 1;
    }

    dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest == -1)
    {
        perror("open destination");
        close(src);
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```

if (dest == -1)
{
    perror("open destination");
    close(src);
    return 1;
}

while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
{
    bytes_written = write(dest, buffer, bytes_read);

    if (bytes_written != bytes_read)
    {
        perror("write");
        close(src);
        close(dest);
        return 1;
    }
}

if (bytes_read == -1)
{
    perror("read");
}

/*
 * lseek() demonstration:
 * Move the source file offset back to the beginning.
 */
if (lseek(src, 0, SEEK_SET) == -1)
{
    perror("lseek");
}

```

```

close(src);
close(dest);

printf("File copied successfully using low-level I/O.\n");

return 0;
}

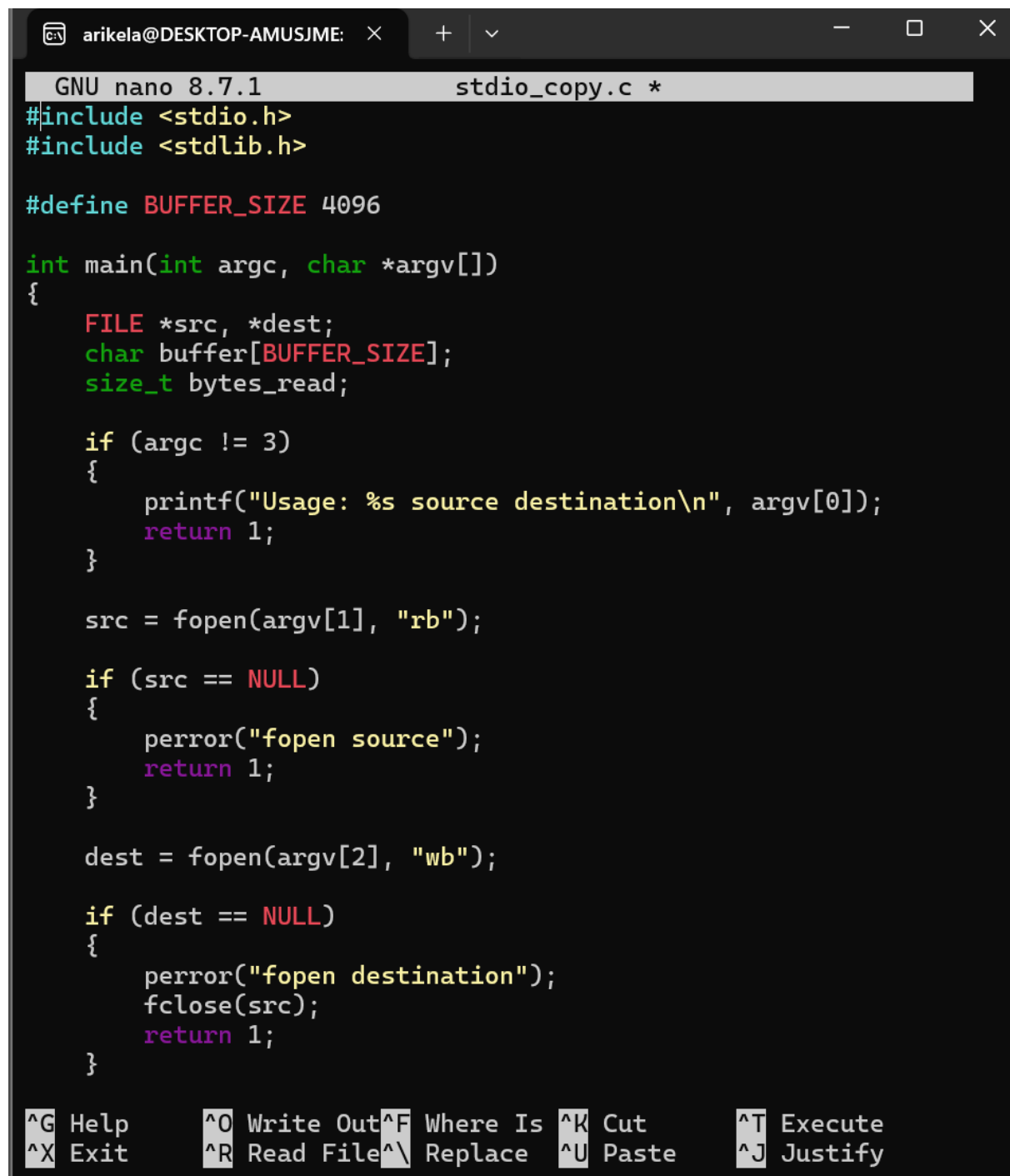
```

Compile and run:

```
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc lowlevel_copy.c -o lowlevel_copy
arikel@DESKTOP-AMUSJME:~/week5prac$ ./lowlevel_copy source.txt destination.txt
File copied successfully using low-level I/O.
arikel@DESKTOP-AMUSJME:~/week5prac$ cat destination.txt
This is a test file for low level Linux file I/O.
```

Standard library:

Stdio_copy.c:



```
GNU nano 8.7.1 stdio_copy.c *
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    if (argc != 3)
    {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    src = fopen(argv[1], "rb");

    if (src == NULL)
    {
        perror("fopen source");
        return 1;
    }

    dest = fopen(argv[2], "wb");

    if (dest == NULL)
    {
        perror("fopen destination");
        fclose(src);
        return 1;
    }

    while (1)
    {
        bytes_read = fread(buffer, 1, BUFFER_SIZE, src);
        if (bytes_read == 0)
            break;
        fwrite(buffer, 1, bytes_read, dest);
    }

    fclose(src);
    fclose(dest);

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
{
    if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
    {
        perror("fwrite");
        fclose(src);
        fclose(dest);
        return 1;
    }
}

fclose(src);
fclose(dest);

printf("File copied successfully using standard I/O.\n");

return 0;
}

```

Compile and run:

```

arikel@DESKTOP-AMUSJME:~/week5prac$ nano stdio_copy.c
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc stdio_copy.c -o stdio_c
opy
arikel@DESKTOP-AMUSJME:~/week5prac$ ./stdio_copy source.txt std
io_destination.txt
File copied successfully using standard I/O.
arikel@DESKTOP-AMUSJME:~/week5prac$ cat stdio_destination.txt
This is a test file for low level Linux file I/O.

```

```

arikel@DESKTOP-AMUSJME:~/week5prac$ dd if=/dev/zero of=largefil
e.bin bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.12735 s, 826 MB/s
arikel@DESKTOP-AMUSJME:~/week5prac$ time ./lowlevel_copy largef
ile.bin lowlevel_large.bin
File copied successfully using low-level I/O.

real    0m0.132s
user    0m0.009s
sys     0m0.121s
arikel@DESKTOP-AMUSJME:~/week5prac$ time ./stdio_copy largefile
.bin stdio_large.bin
File copied successfully using standard I/O.

real    0m0.161s
user    0m0.028s
sys     0m0.133s

```

Conclusion:

Low-level Linux I/O uses file descriptors and system calls such as `open()`, `read()`, `write()`, `lseek()`, and `close()`. Standard I/O uses `FILE *` streams and functions such as `fopen()`, `fread()`, `fwrite()`, and `fclose()`. Low-level I/O provides more direct control over file descriptors, while standard I/O provides a higher-level interface and buffering. Actual performance depends on buffer size, filesystem caching, storage hardware, and workload.

Task-2:

Develop a program that redirects standard input and Output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

Ans:

`redirect_io.c`

arikela@DESKTOP-AMUSJME: × + ▾

GNU nano 8.7.1 redirect_io.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int input_fd;
    int output_fd;

    // Open input file
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1)
    {
        perror("open input");
        return 1;
    }

    // Open output file
    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd == -1)
    {
        perror("open output");
        close(input_fd);
        return 1;
    }

    // Redirect standard input
    if (dup2(input_fd, STDIN_FILENO) == -1)
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```

// Redirect standard input
if (dup2(input_fd, STDIN_FILENO) == -1)
{
    perror("dup2 stdin");
    close(input_fd);
    close(output_fd);
    return 1;
}

// Redirect standard output
if (dup2(output_fd, STDOUT_FILENO) == -1)
{
    perror("dup2 stdout");
    close(input_fd);
    close(output_fd);
    return 1;
}

close(input_fd);
close(output_fd);

// Read from redirected stdin and write to redirected stdout
char buffer[1024];
ssize_t n;

while ((n = read(STDIN_FILENO, buffer, sizeof(buffer))) > 0)
{
    write(STDOUT_FILENO, buffer, n);
}

return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

arikel@DESKTOP-AMUSJME:~/week5prac$ echo "Hello from redirected
standard input!" > input.txt
arikel@DESKTOP-AMUSJME:~/week5prac$ cat input.txt
Hello from redirected standard input!
arikel@DESKTOP-AMUSJME:~/week5prac$ nano redirect_io.c
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc redirect_io.c -o redire
ct_io
arikel@DESKTOP-AMUSJME:~/week5prac$ ./redirect_io
arikel@DESKTOP-AMUSJME:~/week5prac$ cat output.txt
Hello from redirected standard input!
arikel@DESKTOP-AMUSJME:~/week5prac$

```